

DSBD skill 3.2

Code:

CREATE DATABASE BankDB;

USE BankDB;

-- =====

-- CREATE TABLES

-- =====

CREATE TABLE Customer (

Customer_ID INT PRIMARY KEY,

Customer_Name VARCHAR(100) NOT NULL,

Phone VARCHAR(15),

Email VARCHAR(100),

City VARCHAR(50)

);

CREATE TABLE Account (

Account_No INT PRIMARY KEY,

Customer_ID INT,

Account_Type VARCHAR(20),

Balance DECIMAL(12,2),

Branch VARCHAR(50),

FOREIGN KEY (Customer_ID)

REFERENCES Customer(Customer_ID)

);

```
CREATE TABLE Bank_Transaction (  
    Transaction_ID INT PRIMARY KEY AUTO_INCREMENT,  
    Account_No INT,  
    Transaction_Type VARCHAR(20),  
    Amount DECIMAL(12,2),  
    Transaction_Date DATETIME DEFAULT CURRENT_TIMESTAMP,  
    FOREIGN KEY (Account_No)  
        REFERENCES Account(Account_No)  
);
```

```
CREATE TABLE Loan (  
    Loan_ID INT PRIMARY KEY,  
    Customer_ID INT,  
    Loan_Type VARCHAR(30),  
    Loan_Amount DECIMAL(12,2),  
    Interest_Rate DECIMAL(5,2),  
    FOREIGN KEY (Customer_ID)  
        REFERENCES Customer(Customer_ID)  
);
```

```
-- =====  
-- INSERT DATA  
-- =====
```

```
INSERT INTO Customer  
VALUES  
(101, 'Ravi Kumar', '9876543210', 'ravi@gmail.com', 'Hyderabad'),
```

```
(102, 'Priya Sharma', '9876543211', 'priya@gmail.com', 'Vijayawada'),  
(103, 'Arjun Reddy', '9876543212', 'arjun@gmail.com', 'Bangalore'),  
(104, 'Sneha Rao', '9876543213', 'sneha@gmail.com', 'Chennai'),  
(105, 'Kiran Kumar', '9876543214', 'kiran@gmail.com', 'Hyderabad');
```

```
INSERT INTO Account
```

```
VALUES
```

```
(10001, 101, 'Savings', 50000, 'Hyderabad'),  
(10002, 102, 'Savings', 75000, 'Vijayawada'),  
(10003, 103, 'Current', 120000, 'Bangalore'),  
(10004, 104, 'Savings', 45000, 'Chennai'),  
(10005, 105, 'Current', 90000, 'Hyderabad');
```

```
INSERT INTO Bank_Transaction
```

```
(Account_No, Transaction_Type, Amount)
```

```
VALUES
```

```
(10001, 'DEPOSIT', 10000),  
(10001, 'WITHDRAW', 5000),  
(10002, 'DEPOSIT', 15000),  
(10003, 'WITHDRAW', 20000),  
(10004, 'DEPOSIT', 5000);
```

```
INSERT INTO Loan
```

```
VALUES
```

```
(501, 101, 'Home Loan', 5000000, 7.5),  
(502, 102, 'Education Loan', 1000000, 6.5),  
(503, 103, 'Car Loan', 800000, 8.2),  
(504, 104, 'Personal Loan', 500000, 10.5),
```

```
(505, 105, 'Home Loan', 4000000, 7.2);
```

```
-- =====
```

```
-- SIMPLE VIEW
```

```
-- =====
```

```
CREATE VIEW Customer_View AS
```

```
SELECT *
```

```
FROM Customer;
```

```
SELECT * FROM Customer_View;
```

```
-- =====
```

```
-- VIEW WITH CONDITIONS
```

```
-- =====
```

```
CREATE VIEW High_Balance_View AS
```

```
SELECT
```

```
    Account_No,
```

```
    Customer_ID,
```

```
    Account_Type,
```

```
    Balance
```

```
FROM Account
```

```
WHERE Balance > 100000;
```

```
SELECT * FROM High_Balance_View;
```

```
-- =====
```

```
-- VIEW USING JOIN
```

```
-- =====
```

```
CREATE VIEW Customer_Account_View AS
```

```
SELECT
```

```
    C.Customer_ID,
```

```
    C.Customer_Name,
```

```
    C.City,
```

```
    A.Account_No,
```

```
    A.Account_Type,
```

```
    A.Balance,
```

```
    A.Branch
```

```
FROM Customer C
```

```
JOIN Account A
```

```
ON C.Customer_ID = A.Customer_ID;
```

```
SELECT * FROM Customer_Account_View;
```

```
-- =====
```

```
-- AGGREGATE VIEW
```

```
-- =====
```

```
CREATE VIEW Total_Bank_Balance AS
```

```
SELECT
```

```
SUM(Balance) AS Total_Balance,  
AVG(Balance) AS Average_Balance,  
MAX(Balance) AS Maximum_Balance,  
MIN(Balance) AS Minimum_Balance  
FROM Account;
```

```
SELECT * FROM Total_Bank_Balance;
```

```
-- =====  
-- GROUP BY VIEW  
-- =====
```

```
CREATE VIEW Branch_Account_Count AS  
SELECT  
    Branch,  
    COUNT(*) AS Number_of_Accounts,  
    SUM(Balance) AS Total_Balance  
FROM Account  
GROUP BY Branch;
```

```
SELECT * FROM Branch_Account_Count;
```

```
-- =====  
-- HAVING VIEW  
-- =====
```

```
CREATE VIEW Rich_Branches AS
SELECT
    Branch,
    SUM(Balance) AS Total_Balance
FROM Account
GROUP BY Branch
HAVING SUM(Balance) > 100000;
```

```
SELECT * FROM Rich_Branches;
```

```
-- =====
-- TRANSACTION VIEW
-- =====
```

```
CREATE VIEW Deposit_Transaction_View AS
SELECT *
FROM Bank_Transaction
WHERE Transaction_Type = 'DEPOSIT';
```

```
SELECT * FROM Deposit_Transaction_View;
```

```
-- =====
-- ORDER BY VIEW
-- =====
```

```
CREATE VIEW Balance_Ranking_View AS
```

```
SELECT
    Account_No,
    Customer_ID,
    Account_Type,
    Balance
FROM Account
ORDER BY Balance DESC;
```

```
SELECT * FROM Balance_Ranking_View;
```

```
-- =====
-- CALCULATED COLUMN VIEW
-- =====
```

```
CREATE VIEW Loan_Interest_View AS
SELECT
    Loan_ID,
    Customer_ID,
    Loan_Type,
    Loan_Amount,
    Interest_Rate,
    Loan_Amount * Interest_Rate / 100 AS Annual_Interest
FROM Loan;
```

```
SELECT * FROM Loan_Interest_View;
```

-- =====

-- QUERYING A VIEW

-- =====

SELECT *

FROM High_Balance_View

WHERE Balance > 120000;

SELECT *

FROM Customer_Account_View

ORDER BY Balance DESC;

-- =====

-- UPDATE THROUGH VIEW

-- =====

UPDATE Account

SET Balance = 60000

WHERE Account_No = 10001;

SELECT *

FROM Account

WHERE Account_No = 10001;

-- =====

-- INSERT THROUGH VIEW

-- =====

CREATE VIEW Simple_Account_View AS

SELECT

Account_No,

Customer_ID,

Account_Type,

Balance,

Branch

FROM Account;

INSERT INTO Simple_Account_View

VALUES

(10006, 101, 'Savings', 55000, 'Hyderabad');

SELECT * FROM Account;

-- =====

-- DELETE THROUGH VIEW

-- =====

DELETE FROM Simple_Account_View

WHERE Account_No = 10006;

SELECT * FROM Account;

```
-- =====
```

```
-- VIEW INFORMATION
```

```
-- =====
```

```
SHOW FULL TABLES
```

```
WHERE TABLE_TYPE = 'VIEW';
```

```
SHOW CREATE VIEW Customer_Account_View;
```

```
DESCRIBE Customer_Account_View;
```

```
-- =====
```

```
-- DROP VIEWS
```

```
-- =====
```

```
DROP VIEW Customer_View;
```

```
DROP VIEW
```

```
    High_Balance_View,
```

```
    Customer_Account_View,
```

```
    Total_Bank_Balance,
```

```
    Branch_Account_Count,
```

```
    Rich_Branches,
```

```
    Deposit_Transaction_View,
```

```
    Balance_Ranking_View,
```

```
    Loan_Interest_View,
```

```
    Simple_Account_View;
```

```
ON class.id = class_info.id;
```

```
-- INNER JOIN with selected columns
```

```
SELECT class.name, class_info.address
```

```
FROM class
```

```
INNER JOIN class_info
```

```
ON class.id = class_info.id;
```

```
-- NATURAL JOIN
```

```
SELECT *
```

```
FROM class
```

```
NATURAL JOIN class_info;
```

```
-- Add more unmatched records
```

```
INSERT INTO class VALUES
```

```
(5, 'ashish');
```

```
INSERT INTO class_info VALUES
```

```
(7, 'NOIDA'),
```

```
(8, 'PANIPAT');
```

```
-- LEFT OUTER JOIN
```

```
SELECT *
```

```
FROM class
```

```
LEFT OUTER JOIN class_info  
ON class.id = class_info.id;
```

```
-- LEFT JOIN - unmatched records from class
```

```
SELECT *  
FROM class  
LEFT JOIN class_info  
ON class.id = class_info.id  
WHERE class_info.id IS NULL;
```

```
-- RIGHT OUTER JOIN
```

```
SELECT *  
FROM class  
RIGHT OUTER JOIN class_info  
ON class.id = class_info.id;
```

```
-- RIGHT JOIN - unmatched records from class_info
```

```
SELECT *  
FROM class  
RIGHT JOIN class_info  
ON class.id = class_info.id  
WHERE class.id IS NULL;
```

```
-- FULL OUTER JOIN using UNION
```

```
SELECT *  
FROM class  
LEFT JOIN class_info  
ON class.id = class_info.id
```

UNION

```
SELECT *  
FROM class  
RIGHT JOIN class_info  
ON class.id = class_info.id;
```

-- FULL OUTER JOIN - unmatched records only

```
SELECT *  
FROM class  
LEFT JOIN class_info  
ON class.id = class_info.id  
WHERE class_info.id IS NULL
```

UNION

```
SELECT *  
FROM class  
RIGHT JOIN class_info  
ON class.id = class_info.id  
WHERE class.id IS NULL;
```

```
-- =====
```

```
-- SET OPERATORS
```

```
-- =====
```

```
CREATE TABLE first_table (
```

```
    id INT,
```

```
    name VARCHAR(30)
```

```
);
```

```
CREATE TABLE second_table (
```

```
    id INT,
```

```
    name VARCHAR(30)
```

```
);
```

```
INSERT INTO first_table VALUES
```

```
(1, 'abhi'),
```

```
(2, 'adam');
```

```
INSERT INTO second_table VALUES
```

```
(2, 'adam'),
```

```
(3, 'chester');
```

```
-- UNION
```

```
SELECT *
```

```
FROM first_table
```

UNION

SELECT *

FROM second_table;

-- UNION with selected column

SELECT name

FROM first_table

UNION

SELECT name

FROM second_table;

-- UNION ALL

SELECT *

FROM first_table

UNION ALL

SELECT *

FROM second_table;

-- COUNT rows using UNION ALL

SELECT COUNT(*)

```
FROM (  
    SELECT *  
    FROM first_table  
  
    UNION ALL  
  
    SELECT *  
    FROM second_table  
) AS A;
```

```
-- INTERSECT  
SELECT *  
FROM first_table
```

```
INTERSECT
```

```
SELECT *  
FROM second_table;
```

```
-- EXCEPT (equivalent of MINUS)  
SELECT *  
FROM first_table
```

```
EXCEPT
```

```
SELECT *
```

```
FROM second_table;
```